

Projet VCub - Rapport

Auteur : Lefrançois Mathias

Cours : Licence 1 - UE Base de données

Date de rendu : 20 mai 2026

Choix de besoins : besoin 1 (Stations) + besoin 2 (Abonnements)

1. Gestion des stations (besoin 1)

1a. Liste des stations filtrée par état (F/M/T)

1b. Détail d'une station + liste des plots

Affiche numéro, nom, capacité, état, numBorne, puis pour chaque plot : état de maintenance, d'utilisation et le numéro du vélo accroché (ou "vide").

1c. Ajout d'une nouvelle station

Validation des entrées (entiers, état F/M). Erreur PostgreSQL gérée (rollback en cas de violation de contrainte).

2. Gestion des abonnements (besoin 2)

2a. Liste des abonnements

2b. Détail d'un abonnement + liste des abonnés

2c. Mise à jour d'un abonnement

Modification des 4 champs (montant, créditTempsBase, tarifHoraire, caution). Entrée vide = conservation de l'ancienne valeur.

3. Gestion des abonnés (besoin 3)

Non réalisé (choix entre besoin 1 et 3 imposé par le sujet)

Le sujet autorise à ne réaliser qu'un seul besoin parmi 1 et 3 ; le besoin 1 a été choisi. La table Emprunt existe néanmoins en BDD pour conserver un modèle relationnel complet.

4. Requêtes

4a. Stations triées par capacité DESC, nom ASC

4b. Nombre d'abonnés par abonnement

LEFT JOIN pour afficher même les abonnements vides (0 abonné).

4c. Stations avec plus de 5 plots disponibles

Critère : etatUtilisationPlot = 'D'.

4d. Stations avec plots M ET F

Implémentée via GROUP BY + HAVING COUNT(DISTINCT etatMaintenancePlot) = 2.

4e. Stations sans aucun vélo en maintenance

NOT EXISTS sur la jointure Plot-Vélo. Avec le seed actuel, les vélos en maintenance ne sont pas accrochés à des plots (cohérence métier) => les 67 stations sont retournées, comportement attendu.

5. Vue

5a. Affichage de V_NbAbonnesParAbonnement

5b. Abonnements avec plus de 1000 abonnés (depuis la vue)

Renvoie 0 ligne avec le seed (30 abonnés au total). La requête et la logique sont correctes.

Notes techniques

- Requêtes paramétrées (%s) systématiquement utilisées dans le code Python, conformément aux exigences anti-injection du sujet.
- Gestion des transactions : commit explicite après INSERT/UPDATE, rollback en cas d'erreur PostgreSQL.
- Données : 67 stations (depuis le fichier Excel fourni), 1100 vélos, 1267 plots, 4 abonnements, 30 abonnés, 15 emprunts.
- Le fichier script_insertion.sql a été généré par un script Python avec seed déterministe (42). Le sujet n'imposant que le fichier .sql en livrable, le script générateur n'est pas inclus dans le rendu.